

GRAPH-GUIDED AND ADAPTIVE KV-CACHE PREFETCHING FOR RETRIEVAL-AUGMENTED GENERATION

Author details withheld for supervisor review

ABSTRACT

The document order and meaning of the chunks required by a retrieval-augmented language model often show relationships. We have investigated whether these relationships can be used to prepare key-value (KV) cache states before a chunk is requested. We represent the chunks as a graph by including their top M semantic neighbours as well as the structural links that exist within the document. For ranking the candidates, we use two-hop retrieval and select a number of chunks up to K . We compare three methods: fixed weights, a globally learned balance, and an online policy which adjusts the balance after observing ranking misses. The evaluation involves an archived simulated-cache study covering six model configurations, a real experiment carried out with Qwen2.5-1.5B using vLLM and LMCache, and answer-quality experiments with Gemini 3.5 Flash-Lite. For the real access trace derived from Hotpot at $K = 10$, offline adaptation increases prediction recall from 80.50% to 95.33% and reduces the ready-cache time to the first token from 56.68 to 51.36 ms as compared to cosine prefetching. With synchronous candidate preparation, the corresponding times are 902.67 and 883.27 ms, whereas they are 73.41 ms when there is no prefetching. In the separate 60-question quality experiments, offline adaptation raises the HotpotQA F1 from 47.94 to 53.83, while cosine performs better on 2WikiMultiHopQA, with an F1 of 48.18 versus 40.53. Altogether, these results demonstrate the advantage of graph-guided prediction on the tested access trace and show that its impact on answer quality varies according to the dataset.

1 INTRODUCTION

Retrieval-augmented generation (RAG) gives a language model access to external passages that are relevant to a question (Lewis et al., 2020). These passages extend the input that the model has to process before it generates its first answer token. The use of previously calculated key and value tensors can reduce the need for repeated processing, on the condition that the cached states match the context in which they are being reused. This results in a selection problem: when both storage space and preparation time are limited, deciding which passages should be prepared first becomes an issue.

Cosine similarity provides a simple benchmark. Once a chunk has been accessed, the prefetcher is able to choose other chunks that have similar embeddings. However, similarity does not capture all kinds of useful relationships. A paragraph can still depend on the one that comes right before it even if the two paragraphs deal with different topics. Moreover, a passage might also be connected to another passage via an intermediate explanation. These observations are the reason for introducing a chunk graph which includes both semantic and positional information.

We look at this approach by means of GraphKV, which is an external candidate-selection layer. GraphKV does not alter the language model’s parameters or its attention mechanism; instead, it decides which chunks to prepare, while the serving system carries out the computation, storage, and reuse of their KV states. The graph includes semantic neighbours chosen on the basis of rank, a structural backbone that connects adjacent units in the document, and weighted paths with at most two edges. A static graph employs a semantic-first ratio of 0.70/0.30. An offline policy learns a global balance based on training accesses. An online policy begins with that balance and then updates it following an instance where the target was not found.

We have evaluated GraphKV along three different lines of inquiry: simulations of multi-model caching over time, live serving performance as measured by vLLM and LMCache on an NVIDIA T4, and the quality of the answers achieved within fixed token budgets. The evaluation concentrates on target prediction accuracy, the overhead associated with preparing the physical cache, and end-to-end task performance. By analysing 30 separate execution configurations together with paired multi-hop QA benchmarks, we highlight both the latency benefits of structural graph prefetching and the system trade-offs that arise from synchronous candidate generation.

2 RELATED WORK

KV-cache serving and reuse. PagedAttention manages KV-cache memory through a paging approach and underlies vLLM (Kwon et al., 2023). LMCache provides a reusable cache layer connected to an inference engine (LMCache Contributors, 2026). These systems supply the storage and execution machinery used by our real experiment. Our candidate-selection rule operates outside that machinery.

Cache reuse also depends on the arrangement of text in a prompt. CacheBlend studies how to combine independently computed chunk states when their texts occur together, including selective recomputation to account for their interaction (Yao et al., 2024). Our measured setup uses exact per-chunk prefix reuse in stock LMCache. It evaluates advance preparation of individual chunk request.

Caching for RAG. RAGCache organizes intermediate states in a knowledge tree and combines cache management with retrieval-aware execution (Jin et al., 2024). Its design illustrates the importance of both retention and execution scheduling. GraphKV instead focuses on ranking candidate chunks from semantic and structural relationships after a current access. Cosine is our experimental baseline; we do not treat it as a default prefetcher built into vLLM or LMCache.

Embeddings and multi-hop evidence. Sentence embedding methods enable efficient similarity comparisons (Reimers & Gurevych, 2019). We use MiniLM embeddings to construct the semantic part of the graph. HotpotQA and 2WikiMultiHopQA supply questions with evidence distributed across passages (Yang et al., 2018; Ho et al., 2020). LongBench includes long-context tasks spanning these datasets and others (Bai et al., 2024). The presence of multi-hop questions motivates examining relationships between passages; it does not, by itself, imply that a two-hop similarity graph will outperform cosine selection.

3 GRAPH SELECTION AND ADAPTATION

3.1 PROBLEM AND REPRESENTATION

Let $\mathcal{C} = \{c_1, \dots, c_n\}$ contain text units, with embedding e_i , document identity d_i , and position p_i . Given a current unit c_i , a policy returns a set $P_K(i)$ of at most K other units. In the cache experiment, the observed next target is y_t , and a prediction succeeds when $y_t \in P_K(i_t)$. In the answer-quality experiment, the policy selects supporting context around a primary sentence retrieved from the question.

The text unit depends on the experiment. The real cache benchmark uses overlapping token chunks from source context records. The API experiment uses the original evidence sentences and document titles provided by the QA dataset. A unit’s position is meaningful only within its corresponding source document or record.

3.2 SEMANTIC AND STRUCTURAL EDGES

For nonzero embeddings, semantic similarity is

$$s_{\text{sem}}(i, j) = \frac{e_i^\top e_j}{\|e_i\|_2 \|e_j\|_2}. \quad (1)$$

Let $S_M(i)$ be the M most similar other units. A semantic edge is admitted if $j \in S_M(i)$ or $i \in S_M(j)$. This symmetrization allows either endpoint to propose a connection. Unlike a fixed similarity threshold, top- M retains the best available neighbours even when all similarities are relatively low.

The structural score is

$$s_{\text{str}}(i, j) = \begin{cases} \frac{1}{1 + |p_i - p_j|}, & d_i = d_j, \\ 0, & d_i \neq d_j. \end{cases} \quad (2)$$

Immediate same-document neighbours are admitted as edges regardless of semantic similarity. Other same-document pairs receive a structural contribution if admitted through the semantic rule. We do not create an all-pairs structural graph. Distances of one, two, and four yield scores of 0.5, 0.333, and 0.2. The added one keeps the expression bounded, while inverse distance gives nearby units more influence without requiring a learned positional model.

For an admitted edge, let $\tilde{s}_{\text{sem}}$ equal the cosine score when the semantic rule is satisfied, and zero otherwise. The weighted edge is

$$w_{ij} = \alpha \tilde{s}_{\text{sem}}(i, j) + \beta s_{\text{str}}(i, j), \quad \alpha, \beta \geq 0, \quad \alpha + \beta = 1. \quad (3)$$

We set $(\alpha, \beta) = (0.70, 0.30)$ as a baseline to give priority in retrieval to semantic similarity while maintaining document adjacency. This keeps semantic similarity dominant while allowing document order to affect ranking. For example, an adjacent semantic neighbour with cosine 0.8 receives $0.7(0.8) + 0.3(0.5) = 0.71$. A cross-document neighbour with the same cosine receives 0.56. An adjacency-only edge receives 0.15.

The real cache experiment uses $M = 20$; the API experiments use the runner’s $M = 5$ default. These values control candidate coverage in different graph sizes. The fixed adjacency retains at most 100 outgoing neighbours. The adaptive implementation ranks paths from the stored raw edge features before this fixed-weight pruning. Thus its candidate set need not be identical to the fixed policy’s set. The final prefetch or selection budget remains K in every policy.

3.3 TWO-HOP RANKING

The fixed graph considers direct neighbours and neighbours of those neighbours. For a two-edge path with weights a, b , it uses

$$B(a, b) = 0.9 \min(a, b) + 0.1 \max(a, b). \quad (4)$$

The weaker edge contributes most of the score, while the stronger edge breaks ties between paths with the same bottleneck. A path with edge weights 0.8 and 0.6 receives 0.62. We fix 0.9/0.1 coefficients as the bottleneck interpolation weights across all experiments without tuning. For each candidate, the largest score over available paths is retained; candidates are then sorted and the top K returned.

For adaptation, each edge retains a two-component feature $(\tilde{s}_{\text{sem}}, s_{\text{str}})$. Equation 4 is applied separately to each component along a two-hop path, producing $\phi_{ij}^{(r)}$. The adaptive candidate score is

$$q_{\theta}(i, j) = \max_{r \in \mathcal{P}_{ij}} \theta^{\top} \phi_{ij}^{(r)}, \quad \theta = (\alpha, \beta), \quad (5)$$

where $\mathcal{P}_{ij}$ contains one- and two-edge paths. Feature-wise blending followed by weighting is not generally identical to blending already weighted edges. We specify both procedures because they are the actual fixed and adaptive ranking rules.

3.4 OFFLINE LEARNING

The offline policy searches $\alpha \in \{0, 0.05, \dots, 1\}$ with $\beta = 1 - \alpha$. The step gives 21 interpretable choices and makes the small search easy to repeat. In the real cache experiment, the objective is average target recall over the training accesses and tested K values:

$$\theta^* = \arg \max_{\theta} \frac{1}{|\mathcal{T}||\mathcal{K}|} \sum_{t \in \mathcal{T}} \sum_{K \in \mathcal{K}} \mathbf{1}[y_t \in P_K(i_t; \theta)]. \quad (6)$$

Ties favour a semantic weight closer to 0.70. The selected weights are frozen for test evaluation. This is one global pair; the policy does not receive a semantic, structural, or multi-hop query label at test time.

In the API experiment, the objective for fitting was to achieve recall of the supporting sentence when answering the training questions, before the final truncation of the context. When training scores tie, feedback-target hit rate is used as the secondary criterion. Separate weights were determined for each dataset. In the earlier simulation a distinct, adaptive policy that was conditioned on the query type was employed; this is indicated in the results by the fact that its values are archived.

3.5 ONLINE LEARNING

The online policy starts from θ^* and predicts before observing the target. If the target was selected, the weights stay unchanged. If it was absent from the graph’s candidate universe, the policy records a coverage miss; changing weights cannot introduce a missing path. Otherwise, the event is a ranking miss. Let z_t be the highest-ranked selected candidate with stored features. The update uses

$$g_t = \phi(y_t) - \phi(z_t), \quad (7)$$

$$\hat{g}_t = \begin{cases} g_t / \|g_t\|_2, & \|g_t\|_2 > 10^{-12}, \\ g_t, & \text{otherwise,} \end{cases} \quad (8)$$

$$\theta_{t+1} = \Pi_{\Delta}(\theta_t + \eta \hat{g}_t), \quad (9)$$

where Π_{Δ} is Euclidean projection onto the nonnegative, sum-to-one simplex. The size of the correction is controlled by normalisation and the weights are kept interpretable through projection. The constant 10^{-12} is used in order to prevent division by a value which is numerically identical to zero. The updates influence only future events.

The learning rate is selected on development data from $\{0.01, 0.05, 0.1, 0.2, 0.5, 1.0\}$. This spans small adjustments and more aggressive responses. The real Qwen run selects $\eta = 0.01$ and starts from $(0.45, 0.55)$. For a simple illustration, a normalized correction $(0.7071, -0.7071)$ moves that pair to approximately $(0.4571, 0.5429)$. A positive semantic correction therefore shifts a small amount of weight toward semantic evidence.

4 EXPERIMENTAL DESIGN

4.1 ARCHIVED SIX-MODEL SIMULATION

The earlier study supplies simulated-cache results for Qwen2.5-1.5B, 3B, and 7B, Llama-3.2-1B and 3B, and Gemma-3-1B. The Qwen and Gemma summaries pool HotpotQA, 2WikiMultiHopQA, MuSiQue, and MultiFieldQA-en; Llama-1B pools MuSiQue and MultiFieldQA-en, and Llama-3B covers HotpotQA. We report the archived means at $K = 10$ in the main text and the complete available budget sweep in Appendix A. The historical adaptive column refers to the earlier query-type-conditioned policy. Timing is the archived hit-aware simulated-cache quantity, distinct from the directly measured vLLM timing below.

4.2 REAL vLLM AND LMCACHE EXPERIMENT

The actual experiment involves running Qwen2.5-1.5B-Instruct on one NVIDIA Tesla T4 within Kaggle, with only GPU 0 being accessible to child processes and both tensor and pipeline parallelism set at one; the embeddings are generated on the CPU. The experiment makes use of a 0.5 GB LMCACHE capacity, has a GPU memory utilization of 0.82, a maximum sequence length of 4096, and 16-token engine/cache blocks.

The workload consists of 100 context records derived from Hotpot, with each record limited to 16,000 characters and divided into chunks of 384 tokens with an overlap of 64 tokens, resulting in a total of 1,223 chunks. The record identities are divided such that 60 are assigned to the training set, 20 to the development set and 20 to the test set. Each of these sets includes 600 controlled accesses, which are distributed equally among the semantic, structural, and multi-hop categories. These categories refer to the access patterns that have been generated. In this context, document identity means the source

context record and must not be confused with the original Wikipedia-title boundaries that are kept by the API experiment.

The following five policies are assessed for each value of K at $K \in \{6, 10, 16\}$, namely no prefetch, cosine, fixed graph, offline global, and online warm. In each case, two repetitions are carried out using a different random order of the arms. An arm refers to a combination of one policy, one budget, and one repetition, so that there are $5 \times 3 \times 2 = 30$ experimental arms and 18,000 target-access observations. For each policy–budget mean there are 1,200 observations obtained from two runs of the same 600-event trace. The repetitions are included to measure the variability of execution; they do not result in 1,200 independent questions.

Execution path. The GraphKV driver sorts the chunk IDs and then forwards the candidate-population requests to vLLM via its HTTP completion interface. Each selected chunk is processed as its own exact prompt with a one-token completion. The LMCache connector saves the resulting states. When a subsequent target request occurs, it uses the matching prompt identity to obtain those states. The integration makes use of the standard exact-prefix caching and does not include any graph-prefetch setting in vLLM. The candidate population involves actual inference, not merely the transfer of a previously computed cache block.

Before the main experiment, the control procedures look at cold access in comparison with repeated access, wrong-candidate retrieval, known-target retrieval, prompt-token identity, and cache eviction. The oracle control is allowed to prepare the known target in advance and checks that the cache path works as it should. Nevertheless, it is not included as one of the deployable policies in the main comparison. All of the 30 arms have now been completed and the post-run report issued indicates that both the publication controls and the order-independence check have passed.

4.3 GEMINI ANSWER-QUALITY EXPERIMENT

The API study uses the reported model identifier `gemini-3.5-flash-lite`. HotpotQA and 2WikiMultiHopQA each contribute 120 training, 40 development, and 60 held-out validation questions. Training and development are disjoint samples from the training split. The two datasets are processed independently. All four selection policies answer the same 60 questions at $K = 10$, producing 240 completed policy evaluations per dataset. Training and development use supporting-fact labels and require no generated answers.

The sentence which is most similar to the question embedding is known as the primary sentence, and the cosine and graph policies then rank the candidates in relation to this primary sentence. This primary sentence is added to the list of selected sentences. The runner sets a limit on the reference context at 768 tokens according to the Qwen2.5-1.5B tokenizer and cuts down all four contexts to the same number of reference tokens per question, even if that number is less than 768; the token counts for the provider are recorded separately. The generation limit is 64 output tokens since the prompt asks for a brief answer based only on the provided reference text. The order of the policies is randomized for each question, successful calls are saved as checkpoints, and identical prompts can reuse a previously saved response.

For online feedback, the runner chooses the labelled supporting sentence with highest question similarity, excluding the primary when another supporting sentence exists. This target is revealed to the update routine only after selection for the current question. Consequently, the API experiment evaluates sequential learning with labelled evidence feedback. It does not treat the generated answer as a reward or claim that this feedback is automatically available in an unlabelled deployment.

5 MEASUREMENTS

Prediction and cache state. Prediction recall refers to the proportion of target IDs that appear in the top- K predictions. With regard to cache residency, a target can stay in the cache from a previous access even if it was not predicted this time. We present the residency and eviction numbers from the byte-LRU audit model as shadow-cache metrics and the retrieved-token counter from LMCache as a separate observed system metric. The current-prefetch hit rate assigns attribution to the current population step. The estimated prefetch MiB indicates the KV size of the candidate preparation, not a hardware bus-traffic counter.

Table 1: Archived simulated-cache means at $K = 10$ (ms). N is the number of observations per reported policy mean. Adaptive is the historical query-type policy.

Model	No prefetch	Cosine	Fixed graph	Adaptive	N
Qwen2.5-1.5B	31.932	11.391	11.872	7.503	2,400
Qwen2.5-3B	37.120	13.360	14.157	8.992	2,400
Qwen2.5-7B	48.269	17.552	18.001	11.283	2,400
Llama-3.2-1B	23.178	8.270	8.549	5.542	1,200
Llama-3.2-3B	55.425	22.406	22.555	–	600
Gemma-3-1B	40.575	14.891	15.855	9.803	2,400

Latency. Ready-cache TTFT measures the time to the first generated token after candidate preparation finishes. Service time measures the full target request. Population time is the total duration of candidate preparation, and policy time is CPU ranking time. The end-to-end accounting is

$$T_{\text{E2E}} = T_{\text{policy}} + T_{\text{population,exposed}} + T_{\text{ready}}. \quad (10)$$

The real run sets overlap to zero, so all population time is exposed. No-prefetch performs no candidate population. We report both timing boundaries because a policy may make the later request faster while still requiring substantial preparation.

Answer quality. Exact match (EM) is the proportion of predictions that exactly match an accepted reference answer after normalization. Token F1 is the harmonic mean of token precision and recall, and it takes the highest score from among the reference answers. Supporting-fact recall gives the proportion of the labelled evidence sentences that are still completely included after context truncation. A target hit shows whether the specified feedback sentence is completely delivered. These metrics are used to assess text selection and do not involve the evaluation of a Gemini KV-cache hit.

Aggregation and uncertainty. In each experiment the means are calculated and the relevant sample number is kept. The API differences are given as the observed differences between the paired means. Statistical significance requires paired question-level analysis; aggregate means alone do not establish it. The historical FLOP figures are analytical estimates of computation, rather than measurements from a GPU profiler. For the actual vLLM run no reduction in the measured FLOPs is claimed.

6 RESULTS

6.1 SIX-MODEL SIMULATED-CACHE RESULTS

Table 1 summarizes the archived measurements at $K = 10$. Across the five model configurations with an adaptive result, the historical adaptive policy has a lower mean simulated-cache latency than cosine. The relative reduction is approximately 32.7–35.7%. Fixed graph is slightly slower than cosine at this budget in all six configurations, showing that graph construction alone does not reproduce the adaptive result. The model-specific averages retain their original dataset pools; they are not a same-corpus hardware scaling comparison.

The archived adaptive hit rate is 78.21% for each Qwen scale and Gemma-1B, and 77.08% for Llama-1B. Repeated hit rates across model sizes reflect shared selection behaviour and traces; the latency quantities vary with the model configuration. The full budget tables are provided to show the shape of the archived comparisons.

6.2 REAL TARGET PREDICTION AND READY-CACHE LATENCY

The real offline search selects semantic/structural weights of 0.45/0.55, with mean training prediction recall of 90.89%. This is more structural than the fixed 0.70/0.30 reference. Table 2 gives the complete target-recall and ready-cache TTFT results. Every graph-family policy has higher recall and lower ready-cache TTFT than cosine at all three tested budgets.

At $K = 10$, offline adaptation increases recall by 14.83 percentage points and lowers ready-cache TTFT by 5.32 ms, or 9.38%, relative to cosine. Fixed graph reaches 89.50% recall and 53.22 ms.

Table 2: Real Qwen2.5-1.5B results. R is prediction recall (%); T is ready-cache TTFT (ms). Each policy–budget mean contains 1,200 repeated trace observations.

Policy	$K = 6$		$K = 10$		$K = 16$	
	R	T	R	T	R	T
No prefetch	0.00	73.19	0.00	73.41	0.00	72.16
Cosine	68.50	59.78	80.50	56.68	85.33	55.67
Fixed graph	74.83	56.58	89.50	53.22	92.50	53.37
Offline global	79.33	56.20	95.33	51.36	97.33	52.13
Online warm	75.83	55.85	95.17	52.03	97.33	50.69

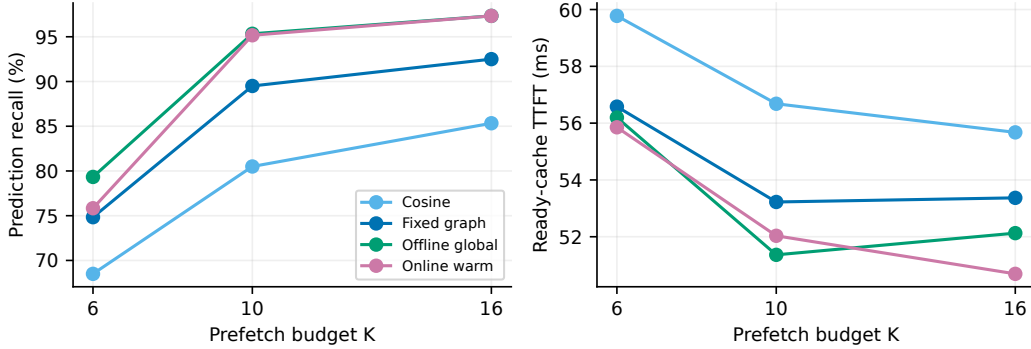


Figure 1: Real Qwen2.5-1.5B results. Left: target prediction recall. Right: ready-cache TTFT, measured after candidate preparation. Each point pools two repetitions of 600 trace events. Lines connect the tested budgets and are not fitted models.

Online warm reaches 95.17% recall and 52.03 ms, close to offline recall but slightly slower in mean ready-cache TTFT. At $K = 16$, online warm has the lowest ready-cache mean, 50.69 ms, and matches offline recall at 97.33%.

The online update frequency declines from 22.50% at $K = 6$ to 3.17% at $K = 10$ and 1.00% at $K = 16$. Larger prediction sets leave fewer ranking misses that trigger updates. Online learning therefore provides a mechanism to respond to observations but does not consistently outperform offline learning.

6.3 PREPARATION COST AND CACHE BEHAVIOUR

Table 3 includes population time in the timing boundary. At $K = 10$, offline adaptation lowers end-to-end TTFT from cosine’s 902.67 to 883.27 ms. However, no-prefetch takes 73.41 ms because it does no speculative preparation. The measured advantage over cosine is therefore an advantage among the tested preparation policies, not an absolute end-to-end speedup over reactive access.

For offline adaptation at $K = 10$, the shadow-cache residency rate is 96.17%, compared with 83.00% for cosine. Mean LMCACHE retrieved tokens are 369.05 versus 317.40. The token counter provides evidence of actual reuse, while the shadow-cache rate explains the corresponding policy-level state. Appendix C reports the full diagnostic table, including preparation sizes, service time, and update rate.

The larger budgets illustrate the trade-off. Offline recall rises from 95.33% at $K = 10$ to 97.33% at $K = 16$, while mean population time grows from 828.48 to 1354.62 ms. A small increase in target coverage can therefore require considerably more work. Whether that preparation is worthwhile depends on when it can run and how often its output is reused.

Table 3: End-to-end TTFT (ms), including policy time and all synchronous candidate-population time.

Policy	$K = 6$	$K = 10$	$K = 16$
No prefetch	73.20	73.41	72.17
Cosine	557.15	902.67	1428.38
Fixed graph	552.29	891.05	1411.78
Offline global	550.06	883.27	1410.24
Online warm	551.09	883.36	1406.41

Table 4: Gemini 3.5 Flash-Lite answer quality at $K = 10$, $M = 5$. All metrics are percentages; each policy answers the same 60 questions within a dataset. Support is fully delivered supporting-sentence recall.

Policy	EM	F1	Support	Target hit
<i>HotpotQA</i>				
Cosine	38.33	47.94	63.61	61.67
Fixed graph	43.33	52.72	64.17	61.67
Offline global	45.00	53.83	63.89	60.00
Online warm	41.67	48.51	63.06	58.33
<i>2WikiMultiHopQA</i>				
Cosine	43.33	48.18	68.33	76.67
Fixed graph	33.33	39.24	64.17	66.67
Offline global	35.00	40.53	66.25	66.67
Online warm	35.00	39.91	66.25	66.67

6.4 GEMINI ANSWER QUALITY ON TWO DATASETS

Table 4 reports both completed API experiments. On HotpotQA, fixed graph improves F1 by 4.78 points over cosine, and offline adaptation improves it by 5.89 points. Offline EM is 45.00%, corresponding to 27 exact answers, compared with cosine’s 23. Online warm reaches 25 exact answers and F1 of 48.51, only 0.57 points above cosine.

On 2WikiMultiHopQA, cosine performs best across all four of the reported metrics. It answers 26 questions exactly correctly, whereas the fixed graph method produces 20 and each of the adaptive methods produces 21. Offline adaptation achieves an F1 score that is 1.29 points higher than that of the fixed graph method, but it is still 7.65 points below the score of cosine. The recall of supporting facts for offline adaptation is 66.25%, compared to 68.33% for cosine, and its target-hit rate is 66.67%, as compared to 76.67% for cosine.

The reversal is relevant to the method’s aim. Multi-hop questions don’t always tend to favour the specific paths caused by semantic neighbourhoods and the order within the documents. On Hotpot, the greatest improvements in answer quality happen when the overall amount of evidence recalled is very similar. On 2Wiki, cosine also results in more labelled evidence being retrieved. These findings support the idea of looking at both the ranking and the actual context provided; they do not point to a single cause of the differences in answers.

7 DISCUSSION

The experiments show a trade-off when it comes to predictive KV-cache serving. Even though taking into account both structural and semantic edges leads to better recall in target prediction and reduces the ready-cache TTFT, synchronous candidate computation incurs substantial prefill overhead, negating latency gains under blocking execution. Candidate population dominates end-to-end latency in the synchronous experiment. Future experiments should investigate whether overlapping preparation with useful work or reusing the prepared chunks can reduce this cost. Moreover, graph-guided context selection shows different results on the two datasets: it enhances answer quality compared to cosine on HotpotQA, while cosine works better on 2WikiMultiHopQA.

However, these experiments do not determine whether the difference is due to document structure, candidate coverage, ranking, or the selected context.

Offline adaptation is a useful reference point since it produces one set of interpretable weights without needing a query-type label. Online adaptation meets a different requirement: it is about responding to observations after deployment starts. Its update rule cannot remedy missing graph coverage, and the benefit it gives depends on which informative ranking misses take place. The current results indicate that Offline adaptation achieves the highest HotpotQA answer quality in the API experiment and the lowest mean ready-cache TTFT at $K = 10$ in the vLLM experiment. Online adaptation achieves the lowest mean ready-cache TTFT at $K = 16$.

All the evidence shows that more research should look into both when and which chunk to prepare. A scheduling policy might take into account the cost of the candidate, the probability of reuse, and the available overlap. A routing policy could choose between semantic rankings and graph rankings by using development data. However, these remain subsequent hypotheses; the present experiments assess the evaluated policies directly without incorporating a selector fitted to test outcomes.

8 CONCLUSION

We presented a hybrid chunk graph for candidate preparation and context selection, with fixed, offline-learned, and online-updated semantic–structural weights. The archived simulation supplies results across six model configurations. In the case of the Qwen2.5-1.5B experiment, graph and adaptive policies improve target recall and ready-cache TTFT over cosine across the three tested budgets. At $K = 10$, offline adaptation reaches 95.33% recall and 51.36 ms ready-cache TTFT. The Gemini experiments also include a comparison with regard to answer quality: graph selection improves performance on HotpotQA, while cosine similarity is still more effective on 2WikiMultiHopQA. These results indicate that graph-guided selection can be used as a workload-dependent element of a cache system, taking into account preparation cost and answer quality as well as prediction accuracy.

REFERENCES

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of ACL*, 2024. URL <https://aclanthology.org/2024.acl-long.172/>.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of COLING*, 2020. URL <https://aclanthology.org/2020.coling-main.580/>.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation, 2024. URL <https://arxiv.org/abs/2404.12457>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi: 10.1145/3600006.3613165. URL <https://arxiv.org/abs/2309.06180>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- LMCache Contributors. LMCache: Documentation and source code, 2026. URL <https://docs.lmcache.ai/>. Accessed September 6, 2026.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019. URL <https://aclanthology.org/D19-1410/>.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. URL <https://aclanthology.org/D18-1259/>.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion, 2024. URL <https://arxiv.org/abs/2405.16444>.

A COMPLETE ARCHIVED SIMULATION SWEEP

The following tables reproduce the available mean simulated-cache latencies from the archived summary. Each model retains its own dataset pool. C denotes cosine, G fixed graph, and A the historical query-type adaptive policy. Dashes mean no value was present. These values are not timings from the stock LMCache experiment.

QWEN2.5-1.5B

Dataset pool: 2WikiMQA, HotpotQA, MultiFieldQA-en, MuSiQue.

K	No prefetch	C	G	A
3	31.592	13.975	22.544	19.315
4	31.916	13.177	20.574	16.493
5	31.817	12.858	18.710	13.571
6	31.550	12.219	16.946	11.427
7	30.514	11.461	15.409	10.324
8	31.739	11.846	14.287	9.102
10	31.932	11.391	11.872	7.503
12	31.431	10.395	10.877	7.031
14	31.818	10.395	10.112	6.272
16	31.855	10.512	9.216	5.814

QWEN2.5-3B

Dataset pool: 2WikiMQA, HotpotQA, MultiFieldQA-en, MuSiQue.

K	No prefetch	C	G	A
3	37.283	17.173	28.632	24.470
4	37.584	16.033	25.227	20.242
5	37.028	15.601	22.534	16.854
6	37.668	15.439	20.428	13.683
7	37.180	14.789	18.958	12.413
8	37.232	14.387	17.246	10.849
10	37.120	13.360	14.157	8.992
12	37.189	12.522	12.891	7.990
14	37.463	12.351	11.928	6.840
16	37.167	12.356	11.336	6.272

QWEN2.5-7B

Dataset pool: 2WikiMQA, HotpotQA, MultiFieldQA-en, MuSiQue.

K	No prefetch	C	G	A
3	50.338	23.214	36.822	31.843
4	50.461	21.017	32.553	26.154
5	49.759	20.414	29.166	21.626
6	49.802	19.805	26.344	18.054
7	50.768	19.639	25.637	16.960
8	48.775	18.697	21.677	13.833
10	48.269	17.552	18.001	11.283
12	48.400	16.157	16.343	10.075
14	48.170	15.783	15.086	8.711
16	48.063	15.590	13.931	7.948

LLAMA-3.2-1B

Dataset pool: MultiFieldQA-en, MuSiQue.

K	No prefetch	C	G	A
3	23.724	10.656	17.219	14.612
4	22.652	9.733	14.817	11.550
5	23.033	9.412	13.575	10.006
6	23.201	9.339	12.254	8.525
8	22.739	8.924	10.156	6.735
10	23.178	8.270	8.549	5.542
12	22.505	7.838	7.448	5.050
14	22.704	7.459	7.034	4.326
16	23.011	7.253	6.524	3.933

LLAMA-3.2-3B

Dataset pool: HotpotQA.

K	No prefetch	C	G	A
3	54.003	26.870	45.441	–
4	55.396	25.291	40.555	–
5	55.384	25.104	35.538	–
6	55.343	24.699	32.689	–
8	55.290	23.788	25.671	–
10	55.425	22.406	22.555	–
12	55.480	21.124	19.942	–
14	55.511	21.042	18.399	–
16	55.004	21.278	17.596	–

GEMMA-3-1B

Dataset pool: 2WikiMQA, HotpotQA, MultiFieldQA-en, MuSiQue.

K	No prefetch	C	G	A
3	41.250	19.195	30.620	25.906
4	41.790	18.015	27.897	21.926
5	40.328	16.925	24.164	17.796
6	39.578	16.456	21.514	14.818
8	39.811	15.685	17.967	11.458
10	40.575	14.891	15.855	9.803
12	39.140	13.455	13.765	8.006
14	40.418	13.311	13.063	7.561
16	39.934	13.627	12.235	6.868

B CONFIGURATION AND REPRODUCTION

The source bundle includes the exact pooled Qwen CSV, archived simulation summary, and a transcription of the two supplied API tables. Tables and vector plots are generated from these files by `build_results.py`. The main vLLM configuration uses seed 43, $M = 20$, $K = 6, 10, 16, 600$ accesses per split, and two repetitions. The API commands use seed 42, $M = 5$, $K = 10, 120/40/60$ training/development/test questions, the MiniLM embedding model, and the Qwen reference tokenizer. The 2Wiki extension changes the structured dataset loader; it does not change the ranking formula. All model, dataset, and output identities should be retained when resuming a run.

C COMPLETE REAL-RUN DIAGNOSTICS

The tables below reproduce diagnostic fields from the pooled headline output; the bundled CSV retains every column, including current-prefetch hit rate. Shadow residency and eviction counts originate from the byte-LRU audit model; retrieved tokens originate from LMCACHE counters. MiB values are estimated KV size of completed preparation. Service is the complete target request, and E2E request includes policy and exposed population time.

BUDGET $K = 6$

Policy	Service ms	Population ms	E2E request ms	Policy ms
No prefetch	73.32	0.00	73.32	0.01
Cosine	59.91	497.13	557.29	0.24
Fixed graph	56.68	492.29	552.39	3.41
Offline global	56.30	490.46	550.16	3.40
Online warm	55.96	491.82	551.19	3.41

Policy	KV MiB	Shadow res. %	Reused tokens	Evictions	Update %
No prefetch	0.00	24.00	83.85	0.00	0.00
Cosine	62.24	72.50	278.14	4.88	0.00
Fixed graph	62.35	79.50	302.60	4.68	0.00
Offline global	62.45	82.67	318.32	4.62	0.00
Online warm	62.37	80.33	319.37	4.64	22.50

BUDGET $K = 10$

Policy	Service ms	Population ms	E2E request ms	Policy ms
No prefetch	73.53	0.00	73.53	0.01
Cosine	56.90	845.73	902.89	0.26
Fixed graph	53.33	834.37	891.15	3.45
Offline global	51.46	828.48	883.37	3.42
Online warm	52.13	827.88	883.46	3.45

Policy	KV MiB	Shadow res. %	Reused tokens	Evictions	Update %
No prefetch	0.00	24.00	79.68	0.00	0.00
Cosine	102.73	83.00	317.40	8.62	0.00
Fixed graph	102.72	91.17	353.96	8.26	0.00
Offline global	102.76	96.17	369.05	8.09	0.00
Online warm	102.79	96.00	365.13	8.08	3.17

BUDGET $K = 16$

Policy	Service ms	Population ms	E2E request ms	Policy ms
No prefetch	72.28	0.00	72.29	0.01
Cosine	55.85	1372.45	1428.55	0.26
Fixed graph	53.47	1354.90	1411.89	3.51
Offline global	52.23	1354.62	1410.34	3.50
Online warm	50.79	1352.22	1406.51	3.49

Policy	KV MiB	Shadow res. %	Reused tokens	Evictions	Update %
No prefetch	0.00	24.00	82.22	0.00	0.00
Cosine	163.21	86.17	334.05	14.48	0.00
Fixed graph	161.90	93.17	345.43	14.13	0.00
Offline global	162.15	97.67	365.58	13.98	0.00
Online warm	162.15	97.67	377.12	13.98	1.00